

Skills — Working Drafts

A scaffold for the virtual-lead-developer skill set. Each skill below has the one-liner from the Game Plan, then a set of empty slots to fill in as we brainstorm. The goal is to walk through them one at a time and pin down: who runs it, when, what it asks for, what it actually does step-by-step, what it outputs, and what's still open.

Status: strawman v1 — names + jobs only. Everything below the one-liner is a placeholder to brainstorm.

How to use this doc: pick one skill, fill in the slots together, leave the rest as scaffolding until we get to them. Don't try to design all of them at once — that's the same anti-pattern the Ladder warns against.

The diagnostic (entry point)

`/readiness-check`

One-liner. Point it at your app; it tells you which gate you're at and what's missing. No more than three next moves — the discipline is one net at a time.

Who runs it. *TBD* **When.** *TBD* **Inputs.** *TBD* (URL? repo path? a Q&A interview? all three?) **What it does, step-by-step.** 1. *TBD* 2. *TBD* 3. *TBD*

Output. *TBD* (gate verdict + top 3 missing nets + which skill to run first?) **Open questions.** *TBD*

Gate 1 — Make it robust (MVRP)

Kills the fear: "If I touch it, I'll break it or lose my work." **Earns you:** the ability to change things fearlessly.

`/safety-net` — DRAFTED

One-liner. One-time setup: git workflow + a staging copy + an automated, restore-tested backup. The single move that gets you out of the danger zone — after it, every change is reversible and previewed.

What the builder ends up with (the mental model)

After `/safety-net` runs, you always have **two versions of your app**, plus a backup layer under both:



- **V1** = the published version. The one your users see.
- **V2** = the safe sandbox. Same shape as V1, separate URL. Where you change things before they go live.
- **Workflow:** change V2 → look at it → like it → promote to V1. (This is what `/ship-change` runs.)
- **Backups** = a separate safety net underneath both versions. V1/V2 protects you from *bad code changes*; backups protect you from *data disasters that have nothing to do with code* (user accidentally deletes their account, hack wipes a table, a migration corrupts data).

Both layers belong in `/safety-net` because both fears live in Gate 1, but they protect against different failure modes — don't mistake "I have V2" for "my data is safe." It isn't, until backups + a tested restore exist too.

Relationship to other skills

- `/safety-net` (this one) = **sets up V1 and V2 + backups**. Run once per project.
- `/ship-change` = **runs the V2 → V1 promotion ritual**. Run every time you touch the app.

`/safety-net` builds the road. `/ship-change` is what you drive on it.

Who runs it. The builder (non-technical). The AI does the typing and the heavy lifting; the builder reads, confirms each step, and ends up understanding what's in place.

When. Once per project, before any real changes go live. Re-run only if the stack changes (e.g., you migrate hosts or databases).

Inputs. None upfront — the skill discovers everything in Step 0. The only assumption is **git** (and if it's not set up, Step 1 sets it up).

Design principles. - **Guided checklist, not autonomous agent.** Each step explains → does → confirms → moves on. The builder leaves knowing what's in place and able to operate it. -

Stack-agnostic. Never assumes Vercel / Supabase / Next.js / anything. Discovers the user's stack first, then translates each generic action into specific commands for their tools. - **Skip-what's-done.** If git already exists, don't re-init. If backups already run, just verify them. Don't redo work.

What it does, step-by-step.

Step 0 — Discover the stack. - Ask in plain English: "What's your frontend? Backend? Database? Host?" If the user doesn't know, inspect the repo and propose an answer: *"Looks like you're using Next.js, Supabase for the database, and Vercel for hosting — does that sound right?"* - Output of this step: a `stack.md` written into the project. Every later step (and every other skill) reads from it.

Step 1 — Git baseline. - If no git repo: init one. - If no remote: push to a private GitHub repo. - Tag the current commit as `known-good-v1` — this is the forever-rollback point. - Write a `.gitignore` that excludes `.env`, secrets, and platform junk. - Protect `main` so nothing can push to it directly (changes must come through a separate branch).

Step 2 — Secrets out of code. - Scan the repo for hardcoded API keys, tokens, passwords, connection strings. - For each one found, move it to an environment variable on the user's host (Vercel env vars UI, Netlify env vars, Cloudflare secrets, etc. — based on Step 0). - Confirm the app still runs after the move.

Step 3 — Staging environment. - Stand up a preview / staging deployment using whatever the user's host calls it (Vercel preview deploys, Netlify branch previews, Cloudflare Pages preview, etc.). - Verify staging works by deploying a trivial visible change to it (e.g., add a `STAGING` banner).

Step 4 — Sanitized clone for staging data. - Write a sanitization script that copies the production database and replaces sensitive fields with fake values: emails →

`user_NN@example.com` , names → fake names from a list, payment info → removed, addresses → fake. (See Brainstorm.md → *Notes & Learnings* for the why.) - Point staging at this sanitized clone instead of prod. - Schedule the script to refresh the clone (weekly is fine for v1). - Verify the clone has fake data, not real, before linking it to staging.

Step 5 — Automated backups. - Turn on scheduled backups on the user's database (Supabase has it built in; Firebase via export; self-hosted Postgres via `pg_dump` cron; etc.). - Verify backups are **actually landing somewhere**, not just "configured." Show the user the latest backup file.

Step 6 — Restore test (un-skippable). - Take the most recent backup. - Restore it into a throwaway empty database (NOT live). - Confirm the restored data looks right (row counts, a few sample records). - **A backup you've never restored from is not a backup.** This step is the whole point of the others.

Step 7 — Cheat sheet. - Write `safety-net.md` into the repo containing, in plain English: - How to roll back to `known-good-v1` (the one command) - The staging URL and how to deploy to it - Where backups live and the restore command - How the sanitization script runs and how to refresh it manually - Which env vars exist and where they're set - Commit it. Future-Ava (and future-AI) reads this and operates everything without re-deriving.

Output. - `stack.md` (committed) — what this project is built on. - `safety-net.md` (committed) — the cheat sheet. - Tag `known-good-v1` in git. - Working staging environment with sanitized data. - Verified backups + a verified restore.

Open questions (resolved). - ~~Stack-specific or agnostic?~~ → **Agnostic.** Discover in Step 0. - ~~Prod copy or sanitized clone for staging data?~~ → **Sanitized clone.** - ~~Does it set up git if missing?~~ → **Yes** (Step 1).

Open questions (still). - How sophisticated does the sanitization script need to be for v1? (Probably: just the obvious fields — emails, names, payment info. Sophisticated PII detection is a future upgrade.) - Does the skill enforce branch protection, or just recommend it? (Probably enforce — that's the whole point of "guided procedure.") - What happens if the user's host doesn't support preview deploys natively? (Defer — handle the 80% case first; document the gap for v2.)

Refinement — Step 0 must classify the project, not just the stack

The skill is for **anything** someone is building with AI — not just full apps. A static HTML site doesn't need "staging with a sanitized DB clone"; that's nonsense (there's no DB). Step 0 must

figure out **what kind of project this is** and scale the steps accordingly.

Project types and which steps apply:

Project type	What this looks like	Which steps apply
Static site	HTML/CSS/JS, no backend, no DB	Git + secrets sweep + preview deploy. Skip staging-DB, sanitization, backups (git history IS the backup).
App with backend, no users yet	Has a DB but no real user data	Git + secrets + staging + fresh empty seeded DB + DB backups + restore test. Skip sanitization.
App with real users / payments	Real people, real data	All seven steps including sanitization.
Mobile app (Expo / React Native)	Phone app + web build	All of above + special handling for client-side "secrets" (see Gap #1 below) + EAS Update channels instead of preview URLs (see Gap #2).

Refinement — Sanitization is a user choice, not a default

Step 4 should present three options with drawbacks and let the user pick. Default depends on the project type from Step 0:

Option	What it is	Drawback
Fresh empty + seeded test data	Blank DB, a few fake rows	Staging feels empty; misses bugs that only show at real volume
Copy of prod	Exact duplicate of live DB	Real users' data duplicated → doubled leak risk; can accidentally email/charge real customers from staging
Sanitized clone <i>(default for live apps)</i>	Copy with a script that swaps PII for fakes	Realistic + safe if leaked, but the script is real work to write

Gaps found on real apps (the v2 backlog for /safety-net)

Logged from walking the v1 design through Club Ralley.

1. Client-side "secrets" rule is too naive. React Native / mobile apps bundle

`EXP0_PUBLIC_*` vars *into the app itself* — they ship to every user's phone, so they aren't

really secret. Sentry DSN and PostHog keys are *designed* to be public. The skill's blanket "move all hardcoded keys to env vars" misses the actual rule: **anything that grants write access to your backend (e.g., a Supabase service role key) must never be in the client bundle — only on a server or edge function.** Patch Step 2 to encode this distinction. **Highest priority** (high-risk failure mode, especially on public repos).

2. **Mobile apps need a different "staging" answer.** Vercel preview URLs work for web builds, but a mobile app's staging is EAS Update channels or dev clients — you can't just visit a URL on your phone. Step 3 needs to fork on project type.
3. **Sanitization script generation is hand-waved.** "Write a script that replaces emails and names with fakes" isn't a real instruction for a complex schema (RLS policies, Supabase-managed `auth.users`, custom tables). Two ways to close this: (a) skill inspects the schema and generates a starter script with the obvious PII columns called out, or (b) skill provides a template + a checklist of "columns that look like PII; confirm each." Probably its own sub-skill in v2.
4. **"Restore into a throwaway DB" needs concrete how-to for non-technical users.** For Supabase that means: create a new free project → download backup → upload into new project → verify → delete. A vibe coder has never done that. **Lighter v1 alternative:** check the backup file's integrity (size > 0, valid format, can be opened) instead of a full restore. Less proof but actually doable. Full restore becomes a Gate 2 upgrade.
5. **Multi-deployment projects (one backend, multiple repos).** Ralley has the main app + the founders dashboard, both off the same Supabase. The current skill assumes one app per run. **v1 decision:** scope to a single repo. Document the limitation; multi-deployment is v2.

Implications of the V1/V2 mental model (things to resolve next session)

Adopting V1/V2 as the outcome framing surfaces two questions worth resolving before we move to `/ship-change` :

1. **Steps 3–4 should be reframed around "what data fills V2."** Once V2 is named as a full second version of the app, the next question is what data it reads from. The three options we wrote earlier — *fresh seeded* / *copy of prod* / *sanitized clone* — are three answers to exactly that question. So in the cleanup pass, fold the sanitization choice into Step 3 ("set up V2") rather than treating it as a separate Step 4.
2. **Is V2 persistent or per-change?** Two valid models: - **Persistent V2** — one staging environment always exists, always reflects the latest in-progress work. Easier to grasp, matches Ava's mental model. Pattern: one long-lived `staging` branch. - **Per-change V2** — every change spins up its own fresh V2 (a new branch + preview deploy), then gets thrown

away after merge. Standard pro setup. Harder to mess up because nothing accumulates in V2 between changes.

Persistent is simpler; per-change is more bulletproof. Decision needed before `/ship-change` because the ritual is different in each.

`/ship-change`

One-liner. The guided safe-change loop you run every time you touch the app. Forces the sequence: branch → write a test → make the change → run the QA harness → deploy to staging → look at it → promote to live, rollback one command away.

Who runs it. *TBD* **When.** *TBD* (every single change, or only "real" changes?) **Inputs.** *TBD* (plain-English description of the change?) **What it does, step-by-step.** 1. *TBD* — branch off main 2. *TBD* — write the test for what's about to change 3. *TBD* — make the change (AI does the typing) 4. *TBD* — run the QA harness 5. *TBD* — deploy to staging 6. *TBD* — human eyeball gate 7. *TBD* — promote to live + tag rollback point

Output. *TBD* **Open questions.** *TBD* (how does this interact with `/qa-harness` — does it require one to exist first? what if there are no tests yet?)

Gate 2 — Make it safe to go live (MVGL)

Kills the fear: "It'll break and a customer will lose money, and I won't even know." **Earns you:** the product is safe for strangers transacting real money.

`/go-live-check`

One-liner. A pre-launch and post-launch readiness audit. Turns the Gate-2 net list (error monitoring, uptime alerts, transaction integrity, a security pass) into a pass/fail report against your real app.

Who runs it. *TBD* **When.** *TBD* (right before relaunch? on a cadence after?) **Inputs.** *TBD* **What it does, step-by-step.** 1. *TBD* — confirm error monitoring wired up 2. *TBD* — confirm uptime alerts firing to a real channel 3. *TBD* — audit auth + input validation 4. *TBD* — verify transaction integrity (no silent loss) 5. *TBD* — check terms/privacy if collecting PII or payments

Output. *TBD* (pass/fail report + the fixes for each fail) **Open questions.** *TBD*

/qa-harness

One-liner. Scaffolds the critical-path tests from a plain-English description of your core flows. "Log in → list an item → buy → pay → confirm" becomes a repeatable check that must never break. Manual checklist first, automated smoke tests second.

Who runs it. *TBD* **When.** *TBD* **Inputs.** *TBD* (a description of the critical flows — what should never break) **What it does, step-by-step.** 1. *TBD* — extract the flows from the description 2. *TBD* — generate a manual checklist version first 3. *TBD* — then scaffold automated smoke tests 4. *TBD* — wire the harness into `/ship-change`

Output. *TBD* **Open questions.** *TBD* (Playwright? Cypress? framework-pick or framework-agnostic?)

/emergency-plan

One-liner. The break-glass **emergency plan**, generated for *this* app while it's healthy — a calm, phone-reachable one-page card for the night something breaks: is it just me or everyone → how to get back to working → what to tell users → who to call. Written and rehearsed on a calm day; and when you panic and type the fire into your AI anyway, that same card keeps the AI on the tested path instead of improvising. (*Renamed from `/incident` ; full design locked 2026-06-29 — see the Skills Brainstorm doc.*)

Who runs it. The non-technical builder, on a calm day, once real people depend on the app (Gate 2). Respects the readiness gate — not ready per `/readiness-check` → it turns them around rather than building a card. **When.** Once to author, then **Fire mode** any night something breaks, then a light refresh after each real break; periodic recheck reminder so it doesn't rot. **Inputs.** Reads `stack.md` (the break-list), `safety-net.md` (the *proven* undo it points at), `go-live-check.md` (wired alarms + must-not-break flows), `qa-harness.md` (flows, for the incident→test loop). **What it does, step-by-step.** 1. Checks the gate (readiness + a tested undo from `/safety-net` — **hard gate**), opens reassuring ("breaking is normal"). 2. Derives *this* app's break-list from `stack.md` (+ always a "the app looks fine but data is wrong / records missing" section — the one break with no alarm). 3. Writes a two-page card: is-it-you-or-them → 🚨/🟡 triage → undo → tell users (+ a simple in-app banner) → who-to-call on page 1; per-

class detail on page 2. 4. Fire drill (default, skippable): the AI breaks a sandbox copy, the builder *watches* the undo restore it → the card line reads `rehearsed: <date>`. 5. Gets the card onto the builder's phone (not "done" until confirmed) + writes a Fire-mode "don't improvise, don't touch data, never claim the undo can't work" contract into the project's AI-memory file.

Output. `emergency-plan.md` (durable, in the repo) + a regenerated phone PDF card; feeds `/qa-harness` (every real break becomes a permanent test) and flags any high-stakes class with no alarm back to `/go-live-check`. **Two moats.** (1) **The card *is* the bounded live-mode prompt** — Fire mode keeps the AI a *reader, not a fixer* (enforced three ways), resolving the paradox that your only 2 a.m. tool is an agent. (2) **Proof-not-presence ported to recovery** — the undo is *rehearsed* (AI fires, builder witnesses), not just written down.

Gate 3 — Make it changeable and supportable (MVE)

Kills the fear: "I'm chained to this — I can't change it safely now that it's live, and I can't hand it off." **Earns you:** ship change #5 as calmly as change #1, forever — and walk away.

`/release-foundation`

One-liner. The driving lesson, run once: sets up a release **schedule** for new features. The headline is the calendar — you pick how often you put out new features (every week / 2 weeks / month) and it builds you a **Future Releases chart**: a plain plan of which feature goes out on which date. The big idea it teaches: now that you can change your app safely, you're a bigger operation, so you stop shipping every feature the second it's done — you put it in a **holding pen** (built and proven, but not live yet) and release it on a date.

Who runs it. The builder, once, with the AI in the passenger seat. **When.** Once, at Gate 3, after `/safety-net` + `/ship-change` are in place. Intent-gated — builders who are deliberately staying small are pruned by `/readiness-check` and never sent here. **Inputs.** The features they actually want to build next (the chart is seeded from real wants, never invented to fill a date). **What it does, step-by-step.** 1. **Names the three lanes** — the clearest "why three skills?": something's **broken** → `/ship-change`, out now, never waits; a new **feature** → the release calendar (`/release-foundation` sets it up, `/release` puts it out on its date); a real **emergency** → `/emergency-plan`. Only NEW FEATURES ride the calendar. 2. **Picks a cadence** — how often new features go out (weekly / every 2 weeks / monthly), anchored to a real repeating day. 3. **Builds the Future Releases chart** (`future-releases.md`) — which

feature ships on which date. Looks *forward*. 4. **Installs the holding-pen habit** — a finished feature waits for its date instead of going live the moment it's built; the builder feels it once (queue something instead of shipping it tonight). 5. **Writes the sorter into the everyday AI's memory** — the fix-now-vs-can-wait line that decides, going forward, what's a fix-now (out now via `/ship-change`) versus a new feature (onto the chart). This rides *underneath* the calendar; it's not the headline. 6. **Teaches the one rule that ends the panic** — "miss the train, catch the next one": if a feature isn't ready by its date, it just moves to the next date. No all-nighter.

Output. A chosen cadence, the **Future Releases chart** (`future-releases.md` , looks forward), the sorter rule in AI-memory, a re-run reminder, and a seeded `CHANGELOG.md` (looks backward, kept up to date by `/release`). Dates, not version numbers. **Note.** Its proofs are softer than the other skills' — a schedule has no day-one break to catch; the payoff arrives on real release days. The one renewable check is the re-run health-check that spots a stalled rhythm.

`/release`

One-liner. Release day, run on each release date (or whenever a parked feature is ready). Reads the Future Releases chart, takes the feature(s) parked for today, **catches each one up** to the live app, re-runs the safety checks (`/qa-harness`), the builder takes the real look, then **flips it on** — the genuine go-live moment, *because the feature was held in its holding pen until now* (true even on an app that deploys the instant you save).

Who runs it. The builder, with the AI driving; the builder watches the checks go green, takes the real look, and gives the okay to flip on. **When.** Every release date, or whenever a parked feature is ready. Cadence defined by `/release-foundation` . **Inputs.** The Future Releases chart (what's parked for today), the parked feature branch(es), `/qa-harness` (the checks ARE the seatbelts — no harness, it routes you there first), and `/safety-net` (the tested way back). **What it does, step-by-step.** 1. Locks onto the real live copy of the app (refuses to bless a stale copy). 2. Catches each parked feature **up** to the live app. 3. Re-runs the automated safety checks (`/qa-harness`) — builder watches them go green. **Red check = STOP cold:** no flip-on, no bookmark, no notes; offers to fix via `/ship-change` and come straight back. 4. Builder takes the real look, then **flips the feature on** (the one irreversible, reaches-real-users moment — never without their okay). 5. Saves a recoverable **known-good bookmark** (a named version tag). 6. Writes the plain what-changed list — release notes + changelog: **New** = the features just flipped on; **Fixed** = bug fixes that already went out via `/ship-change` . 7. Updates the

chart — shipped features marked done, anything not ready rolled to the next date ("missed the train, catches the next one").

Output. A proven-good, bookmarked, written-down release: updated `future-releases.md`, a new `CHANGELOG.md` entry (New + Fixed), and a fresh known-good version tag. **Boundary.** Not `/ship-change`. `/ship-change` = one change (a fix goes out now, a feature gets parked). `/release` = putting the parked, scheduled feature(s) live on their date, seatbelts on. New features ride this; bugs/breaks don't wait for it.

`/handoff`

One-liner. Generates the runbooks and docs that make the app supportable by someone other than you — what turns it from a leash into a real asset. The gut-check it answers: *if you vanished for a month, would your app survive, and could someone else keep it alive without calling you?*

Who runs it. The non-technical owner, solo, guided — never autonomous. Two reasons people reach for it: **(1) "I'm handing it off"** — a real person is taking over (contractor, co-founder, friend); and **(2) "I'm insuring myself"** — nobody specific, I just don't want to be the single point of failure. **Ava's call (2026-06-29): support both, but make peace-of-mind the default.**

When. The capstone — and the genuine *end* of the road. You don't reach `/handoff` until `/readiness-check` routes you here, and it only routes you here once you're at the handoff gate with the whole ladder behind you — so `/handoff` can never be run with a rung missing (that's `/readiness-check`'s whole job: diagnose the gate, hand back the next move). Re-runnable, and it installs its own refresh reminders (below) because the binder rots.

Inputs. Mostly *assembles* what earlier skills already wrote — `stack.md` (including the `## Logins & data inventory` `/readiness-check` writes), `safety-net.md`, the `/emergency-plan` card (`emergency-plan.md`), the deploy + rollback procedure from `/ship-change`, and the plain-English what-changed history in `CHANGELOG.md` (a readable retelling of `versions.md`, which `/ship-change` owns — seeded by `/release-foundation`, kept up to date by `/release`). The genuinely new work is the accounts treasure-hunt and the cold-read proof.

What it does, step-by-step. 1. **Ask which reason** — peace-of-mind (default; touches nothing, never locks you out) vs. actually-leaving (opens the gated transfer door at the very end). 2. **The**

accounts inventory — the centerpiece. Discover every account the app leans on; for each card: what it's for · who owns it · *where* the login lives (never the password itself) · what dies if it lapses · is it about to lapse (the time-bombs). On an all-in-one platform (Lovable, v0, Replit) this often *is* the platform's own teammate/secrets panel — note that rather than pointing at a separate tool. 3. **Plain-English architecture map** — one plain "what talks to what" paragraph in words, no boxes-and-arrows diagrams. 4. **Deploy + rollback + recover** — folded from `/ship-change` 's procedure and the `/emergency-plan` card. 5. **"First day for a new operator" guide** — how to log into everything, how to make one tiny safe change via `/ship-change`, and what to do if it breaks (the emergency card). 6. **The cold-read proof** — a memory-less reader (me, or a fresh AI) does one real task — publish a tiny change, or recover from a pretend outage — using *only* the packet; every spot they get stuck is a hole; patch and repeat until it carries someone cold. 7. **(Only if actually-leaving — a separate gated door:)** ownership transfer, in safe order — add the new person (share via the password manager, or, gold standard, invite them as their own user on each account), let them prove they're fully in control from their own account, *then* hand over the primary owner/admin role, remove the old owner's access (revoke their seat, sign out their old sessions, un-share the vault), and replace ("rotate") every key, recovery code, and shared secret the old owner ever saw so the old ones stop working. Includes the domain registrar/DNS account and re-enrolling two-factor login on the new owner's phone. Never remove the old owner until the new one is proven in.

Output. `handoff.md`, committed to the repo, **map-not-vault** (never a real password or key) + a printable/shareable copy for a new owner who has no repo access. Treat that copy as confidential — it's most likely to leak by being emailed, screenshotted, or shared as a Drive link, not by a git breach. Carries a `refreshed_on <date>` stamp because the packet rots.

Open questions. (a) **Where the password manager gets set up — resolved (Ava's call, 2026-06-29):** `/safety-net` sets it up, once and early, alongside getting secrets out of the code; `/readiness-check` only flags its absence, and by the time you reach `/handoff` the manager already exists and `/handoff` just *uses* it. (Trade-off Ava weighed and overrode: the verification pass argued this stretches `/safety-net` 's lean floor; she chose the cleaner handled-once-up-front experience anyway, on condition `/safety-net` keeps it light.) For all-in-one-platform builders, prefer the platform's own teammate/secrets panel rather than a second tool. (b) How the printable/shareable copy is generated — **still genuinely open**. (c) **How the refresh gets nudged — resolved (Ava's call, 2026-06-29): two scheduled reminders.** An automated time-bomb / stale-binder watch that stays silent until something's about to lapse ("*your domain renews in 14 days on a card that expires next month*"), plus a calendar reminder for the full human re-run — both honor map-not-vault + cost-silence (they watch dates and whether a login is *reachable*, never a secret's value). (d) **What happens when an upstream piece is missing — resolved (Ava's call, 2026-06-29): it can't happen by**

design. `/readiness-check` won't route you to `/handoff` unless you're at the handoff gate with the ladder complete, so there's no hole to fabricate around — the router owns the gating, not a marker roll-call inside `/handoff`. (e) **New (2026-06-29): money stays out for now.** The "who pays / what card / what dies if the payment fails" billing-succession line is **parked in BACKLOG.md** — cost-silence holds and the planned **future cost skill** remains money's home; `/handoff` says nothing about dollars for now. And `/handoff` **stays one skill** — the heavy ownership-transfer door stays inside it, behind the gated "I'm actually leaving" path.

Cross-cutting open questions

These apply to more than one skill — worth resolving before going deep on any single one.

1. **Where do skills live?** As markdown files in `~/claude/skills/` ? A separate repo? Bundled with the project they operate on?
 2. **How much does each skill assume about the stack?** (Vercel + Supabase + Next.js by default, then variations?)
 3. **What's the human-gate UX?** A pause in the chat asking for confirmation? A checklist the user signs off on?
 4. **What's reusable across skills?** ("read the project's stack," "find the staging URL," "tag a known-good point" all appear in multiple skills — should they be sub-skills?)
 5. **What's the v1 surface area?** Which skills do we actually build first, vs. which stay as scaffolds in this doc?
-

Next pass

Pick one skill. Fill in its slots. Move to the next. Don't try to design them all in parallel — same atom-at-a-time discipline as the Ladder itself.